CONFIDENTIAL — INTERNAL ENGINEERING REFERENCE — ENHANCED NDA REQUIRED

IOS+ ENGINEERING BODY — DOCUMENT 3 OF 6 — MAY 2026

COS+ Database Schema & Role Architecture

Complete PostgreSQL DDL, Role Definitions, WORM Enforcement Triggers, Partitioning Strategy & Migration Procedures

Document ID:	**EB-DOC-03**
Series:	**IOS+ Engineering Body — Document 3 of 6**
Version:	**1.0**
Date:	**May 2026**
Classification:	**CONFIDENTIAL — Enhanced NDA Required**
Prepared By:	**SMEPro Technologies Engineering**
Contact:	**cmiguez@smeprotech.com**
Audience:	**Database Engineers, Infrastructure Engineers, Security Auditors, NDA-Cleared Technical Evaluators**

Table of Contents

Section 0 — Engineering Body Series Context

This document is part of the IOS+ Engineering Body (EB), a six-document technical specification series produced by SMEPro Technologies Engineering. The table below indexes all documents in the series. **EB Doc 3 is the authoritative schema reference for the COS+ Database.**

Document	Title	Scope	Status
EB Doc 1	IOS+ Internal Architecture	System topology, middleware engine design, connection pool architecture, service boundary definitions	v1.0

Document	Title	Scope	Status
EB Doc 2	Cryptographic Specification	Ed25519 key lifecycle, JCS canonicalization, Evidence Package signing protocol, public key publication	v1.0
EB Doc 3	COS+ Database Schema & Role Architecture **THIS DOCUMENT**	Complete PostgreSQL DDL for all tables, indexes, triggers, roles, and partitioning configuration for the COS+ Database	v1.0
EB Doc 4	Gate 530 Compliance Engine	Dimension evaluation pipeline, threshold model, quarantine escalation logic, regulatory overlay application	v1.0
EB Doc 5	RAG Vault Pipeline	Corpus ingestion, HNSW retrieval queries, RRF fusion, chunk classification and compliance filtering	v1.0
EB Doc 6	Operations Runbook	Deployment procedures, WAL recovery, key rotation, incident response, P1 escalation matrix	v1.0

⚠ SCHEMA DIVERGENCE IS A P1 INCIDENT

EB Doc 3 contains the complete PostgreSQL DDL for all tables, indexes, triggers, roles, and partitioning configuration in the COS+ Database. Engineers modifying the schema **must update this document in the same pull request as the migration.** Any divergence between this document and the live schema

constitutes a P1 incident and must be resolved within the P1 SLA defined in EB Doc 6.

Section 1 — Database Overview & Design Principles

1.1 COS+ as Compliance-Primary Storage

COS+ is not a general-purpose relational database. Its schema is deliberately organized around the **primary query pattern of a compliance-governed AI system**: *"retrieve records matching these compliance dimensions"* — not *"retrieve record with this ID."* Every structural and indexing decision in this schema reflects that governing constraint.

This distinction has concrete engineering consequences. Surrogate primary keys (UUIDs) exist for referential integrity purposes only — they are never the intended access path for compliance queries. The presence of a `WHERE evidence_package_id = $1` clause in application code should be treated as a design smell. Canonical access patterns are compliance-dimension-first, tenant-scoped, and time-bounded.

The three foundational schema design principles that govern every table definition in this document are:

1. **Primary index on `compliance_flags[]` + `classification_level` — not on surrogate ID.** GIN indexes on array columns and composite B-tree indexes on `(classification_level, tenant_id)` are the performance-critical paths. UUID primary key indexes exist for FK constraint enforcement only.

2. **Append-only audit tables — no UPDATE or DELETE on any audit record; enforced at both role and trigger layers.** WORM enforcement is not a policy — it is a structural guarantee enforced by PostgreSQL row-level triggers that fire before any modification reaches storage. This dual-layer enforcement (role privileges + trigger) means no authenticated session, including `cos_admin`, can silently corrupt the audit record.
3. **Compliance metadata is first-class schema — not stored as unstructured JSON blobs alongside a "real" schema.** Fields such as `classification_level`, `compliance_flags[]`, `jurisdiction_flags[]`, and `regulatory_profile_id` are typed, indexed, constraint-validated columns — not keys inside an opaque `metadata JSONB` column. This ensures compliance queries can be executed by the query planner with selectivity estimation and index use; they are not deferred to application-layer JSON parsing.

1.2 PostgreSQL Version & Extensions

PostgreSQL 16.x minimum required. The schema uses declarative partitioning syntax, `INCLUDE` clauses on B-tree indexes, and GIN/HNSW index types that require this version floor. Deployment against an earlier version is a schema compatibility error and will produce migration failures.

The following extensions must be installed before any DDL in this document is executed:

Extension	Purpose	Used By
`pgcrypto`	Provides `gen_random_uuid()` and `digest()` functions used in column defaults and stored procedures	All tables (UUID generation); Evidence Fabric signing service
`vector` (pgvector)	Provides the `vector(1536)` column type and HNSW index operator class (`vector_cosine_ops`)	`objects`, `rag_chunks` — vector embedding columns and HNSW indexes

Extension	Purpose	Used By
`btree_gist`	Enables GiST index support for composite compliance dimension columns in exclusion constraints	Compliance dimension composite indexes
`pgaudit`	Session-level audit logging for `cos_admin` sessions; out-of-band trail	DBA operations audit trail (not schema DDL — operational requirement)

Install all required extensions with the following DDL (must be executed as a superuser before any other schema DDL):

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; CREATE EXTENSION IF NOT EXISTS vector; CREATE EXTENSION IF NOT EXISTS btree_gist;
```

> **ℹ Extension Version Note**
>
> pgvector must be version 0.7.0 or later to support HNSW index builds with the `m` and `ef_construction` storage parameters used in Section 5. Earlier versions support only IVFFlat indexes and are not compatible with this schema.

Section 2 — Role Architecture

2.1 Role Definitions

COS+ uses a minimum-privilege role model. Each role is scoped to a single service or operational function. No role is granted superuser or `CREATEROLE` except `cos_admin`, which is restricted to infrastructure operations and subject to independent session audit logging. Passwords for all roles are injected at deployment time from

HashiCorp Vault; no plaintext passwords appear in version-controlled migration files.

```
-- Application read/write role for IOS+ Middleware Engine CREATE ROLE ios_app
WITH LOGIN PASSWORD '<<injected-by-vault>>'; COMMENT ON ROLE ios_app IS
'Primary application role for IOS+ Middleware Engine. SELECT/INSERT on
operational tables. No access to audit log tables.';  -- Audit write-only
role for Evidence Fabric signing service CREATE ROLE audit_writer WITH LOGIN
PASSWORD '<<injected-by-vault>>'; COMMENT ON ROLE audit_writer IS 'INSERT-
only on all audit log tables. UPDATE and DELETE explicitly revoked. Used
exclusively by the Evidence Fabric service.';  -- Audit read-only role for
compliance review and external auditors CREATE ROLE audit_reader WITH LOGIN
PASSWORD '<<injected-by-vault>>'; COMMENT ON ROLE audit_reader IS 'SELECT-
only on all audit log tables. Used by compliance review API and NDA-cleared
external auditors.';  -- Vector corpus read role for RAG Vault retrieval
service CREATE ROLE rag_reader WITH LOGIN PASSWORD '<<injected-by-vault>>';
COMMENT ON ROLE rag_reader IS 'SELECT-only on vector corpus tables and chunk
metadata. Used by RAG Vault retrieval service.';  -- RAG Vault ingestion
write role CREATE ROLE rag_writer WITH LOGIN PASSWORD '<<injected-by-
vault>>'; COMMENT ON ROLE rag_writer IS 'INSERT/UPDATE on vector corpus and
chunk metadata tables. Used by RAG Vault ingestion pipeline.';  --
Infrastructure admin role — no application use CREATE ROLE cos_admin WITH
LOGIN PASSWORD '<<injected-by-vault>>' CREATEROLE; COMMENT ON ROLE cos_admin
IS 'DBA role for schema migrations and infrastructure operations. All
sessions logged to out-of-band admin audit trail.';
```

2.2 Privilege Matrix

The following table specifies the complete grant model for all table categories. Any privilege not listed is implicitly denied. Explicit REVOKE statements in Section 2.3 formalize denials that require active enforcement beyond the default-deny baseline.

Table / Schema Group	ios_app	audit_writer	audit_reader	rag_reader	rag_writer	cos_admin
Operational tables objects, tenant_registry, ios_signing_keys, regulatory_profiles	SELECT, INSERT	NONE	SELECT	SELECT	NONE	ALL
Audit tables	NONE	INSERT	SELECT	NONE	NONE	ALL

Table / Schema Group	ios_app	audit_writer	audit_reader	rag_reader	rag_writer	cos_admin
evidence_packages, gate_decisions, evidence_source_manifest, quarantine_records, merkle_roots						
Vector corpus tables rag_chunks, rag_sources, rag_partitions	NONE	NONE	NONE	SELECT	INSERT, UPDATE	ALL

> **i Note on cos_admin and WORM**
>
> Although `cos_admin` holds ALL privileges on audit tables, WORM trigger enforcement applies unconditionally — including to `cos_admin` sessions. The trigger function does not interrogate `SESSION_USER` to grant exceptions. The only operations exempt from the trigger are PostgreSQL internal maintenance operations (`VACUUM`, `autovacuum`), which do not touch user row data.

2.3 Explicit REVOKE Statements

Privilege grants alone are not sufficient to model a minimum-privilege system in PostgreSQL. Role inheritance, future grants, and `PUBLIC` defaults can introduce

privilege escalation paths that are invisible without explicit REVOKEs. The following statements are required components of the role model — not optional hardening steps.

```
-- audit_writer: explicitly cannot modify or delete audit records REVOKE
UPDATE, DELETE ON ALL TABLES IN SCHEMA public FROM audit_writer; REVOKE
TRUNCATE ON ALL TABLES IN SCHEMA public FROM audit_writer;  -- audit_reader:
explicitly cannot write anything REVOKE INSERT, UPDATE, DELETE, TRUNCATE ON
ALL TABLES IN SCHEMA public FROM audit_reader;  -- rag_reader: explicitly
cannot write REVOKE INSERT, UPDATE, DELETE, TRUNCATE ON ALL TABLES IN SCHEMA
public FROM rag_reader;  -- ios_app: explicitly cannot touch audit tables
REVOKE ALL ON evidence_packages, gate_decisions, evidence_source_manifest,
quarantine_records, merkle_roots FROM ios_app;
```

> ⚠ **pgaudit Configuration for cos_admin Sessions**
>
> `cos_admin` sessions are recorded to an out-of-band audit trail via the `pgaudit` extension, configured with `log_level = LOG` and `log_relation = on`. This captures every relation-level operation performed under the `cos_admin` role. Critically, `cos_admin` cannot disable `pgaudit` logging without a separate OS-level access to the PostgreSQL configuration files — and that OS-level access is itself independently monitored by the infrastructure security team.

Section 3 — Audit Table Schemas & WORM Enforcement

All tables in this section are append-only. The WORM enforcement trigger defined in Section 3.2 is applied to every table in this section. The role privilege model in Section 2 provides a first layer of write protection; the trigger provides a second, independent layer that fires even when role-level controls are bypassed (e.g., by a future misconfigured grant).

3.1 evidence_packages Table

The `evidence_packages` table is the root audit record for every IOS+ inference cycle that reaches Layer 6 output synthesis. One row per completed inference cycle. Rows for BLOCK decisions are written with null output fields. This table is the primary evidence artifact consumed by compliance review queries, external audit interfaces, and Merkle root computation.

```
CREATE TABLE evidence_packages (     -- Identity     evidence_package_id
UUID PRIMARY KEY DEFAULT gen_random_uuid(),     trace_id                UUID
NOT NULL UNIQUE,      -- Tenant & session     tenant_id               UUID
NOT NULL REFERENCES tenant_registry(tenant_id),     session_id
UUID,      -- Query provenance     query_hash              CHAR(64) NOT NULL,
-- SHA-256 hex     model_fingerprint       VARCHAR(256) NOT NULL,      --
Gate 530 decision     gate_decision_id        UUID NOT NULL REFERENCES
gate_decisions(gate_decision_id),     gate_decision_state     VARCHAR(12) NOT
NULL CHECK (gate_decision_state IN ('PASS', 'BLOCK', 'QUARANTINE')),      --
Output (null for BLOCK)     output_hash             CHAR(64),
-- SHA-256 hex; NULL if BLOCK     confidence_score        NUMERIC(4,3) CHECK
(confidence_score BETWEEN 0 AND 1),     confidence_band         VARCHAR(8)
CHECK (confidence_band IN ('HIGH', 'MEDIUM', 'LOW')),      -- Signing
signing_key_id          UUID NOT NULL REFERENCES ios_signing_keys(key_id),
jcs_canonical_hash      CHAR(64) NOT NULL,      -- SHA-256 of the canonical
bytes that were signed     ed25519_signature       CHAR(88) NOT NULL,      --
Base64url-encoded 64-byte signature      -- Compliance classification
classification_level    SMALLINT NOT NULL CHECK (classification_level BETWEEN
0 AND 4),     compliance_flags        INTEGER[] NOT NULL DEFAULT '{}',    --
Dimension IDs with non-zero scores      -- Temporal anchors (all set at
insert time; never updated)     ingestion_timestamp     TIMESTAMPTZ NOT NULL,
l4_timestamp            TIMESTAMPTZ NOT NULL,     gate_timestamp
TIMESTAMPTZ NOT NULL,     l6_timestamp            TIMESTAMPTZ,            --
NULL if BLOCK     synthesis_timestamp     TIMESTAMPTZ,            -- NULL if
BLOCK     committed_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),      --
Retention     tenant_encryption_key_id UUID NOT NULL,         -- References
key_management_service; for key-shredding erasure     retain_until
TIMESTAMPTZ NOT NULL,   -- Computed from tenant retention profile at insert
time      CONSTRAINT no_future_ingestion CHECK (ingestion_timestamp <=
committed_at + INTERVAL '5 minutes') );  -- Primary index: compliance-first
access pattern CREATE INDEX idx_ep_compliance ON evidence_packages     USING
GIN (compliance_flags)     WHERE gate_decision_state = 'PASS';  CREATE INDEX
idx_ep_classification ON evidence_packages     (classification_level,
tenant_id, committed_at DESC);  CREATE INDEX idx_ep_tenant_time ON
evidence_packages     (tenant_id, committed_at DESC);  CREATE INDEX
idx_ep_gate_decision ON evidence_packages (gate_decision_id); CREATE INDEX
idx_ep_trace ON evidence_packages (trace_id);
```

3.2 WORM Enforcement Trigger on evidence_packages

The `enforce_append_only()` trigger function is defined once and applied to all audit tables. It fires `BEFORE` any `UPDATE` or `DELETE` operation reaches storage and raises a `restrict_violation` exception that aborts the transaction. The exception message captures the operation type, table name, session user, and application name — providing an intrusion-visible audit trail in the PostgreSQL log even when the operation is rejected.

```
-- Trigger function: abort any UPDATE or DELETE on audit tables CREATE OR
REPLACE FUNCTION enforce_append_only() RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN     RAISE EXCEPTION         'WORM VIOLATION: % operation on table % is
prohibited. Audit records are append-only. Attempted by session user: %,
application_name: %',         TG_OP,         TG_TABLE_NAME,
SESSION_USER,         current_setting('application_name')     USING ERRCODE =
'restrict_violation';     RETURN NULL; END; $$;  -- Apply to
evidence_packages CREATE TRIGGER trg_evidence_packages_worm     BEFORE UPDATE
OR DELETE ON evidence_packages     FOR EACH ROW EXECUTE FUNCTION
enforce_append_only();
```

> **i Trigger Scope Notes**
>
> The trigger fires **BEFORE** the operation reaches storage. Even the `cos_admin` role triggers this — the function does not inspect `SESSION_USER` or grant any role-based exception. The only operations exempt are PostgreSQL internal maintenance operations (`VACUUM`, `autovacuum`), which operate at the page level and do not interact with user row triggers.

3.3 gate_decisions Table

The `gate_decisions` table records the output of every Gate 530 evaluation. Each row captures the full dimension score vector, the regulatory profile version active at evaluation time, and the human reviewer decision for QUARANTINE cases that were cleared. Gate decision records are referenced by `evidence_packages` via foreign key; the foreign key constraint is deferrable to support the Evidence Fabric's insert ordering.

```
CREATE TABLE gate_decisions (     gate_decision_id        UUID PRIMARY KEY
DEFAULT gen_random_uuid(),     trace_id                UUID NOT NULL,
```

```
tenant_id               UUID NOT NULL REFERENCES tenant_registry(tenant_id),
-- Decision     decision_state          VARCHAR(12) NOT NULL CHECK
(decision_state IN ('PASS', 'BLOCK', 'QUARANTINE')),     evaluation_timestamp
TIMESTAMPTZ NOT NULL,     evaluation_duration_ms  INTEGER NOT NULL CHECK
(evaluation_duration_ms > 0),      -- Dimension results     dimension_scores
JSONB NOT NULL,     -- Array of {dimension_id, score, threshold, state}
violated_dimensions     INTEGER[] NOT NULL DEFAULT '{}',      --
Configuration at evaluation time     regulatory_profile_id   UUID NOT NULL,
model_version_id        VARCHAR(256) NOT NULL,      -- Human review
(QUARANTINE only)     reviewer_id             UUID,               -- NULL
unless QUARANTINE was human-cleared     reviewer_decision_at    TIMESTAMPTZ,
reviewer_notes          TEXT,      -- Signing     signing_key_id
UUID NOT NULL REFERENCES ios_signing_keys(key_id),     ed25519_signature
CHAR(88) NOT NULL,      committed_at            TIMESTAMPTZ NOT NULL DEFAULT
NOW() );  CREATE INDEX idx_gd_tenant_time ON gate_decisions (tenant_id,
evaluation_timestamp DESC); CREATE INDEX idx_gd_state ON gate_decisions
(decision_state, tenant_id, evaluation_timestamp DESC); CREATE INDEX
idx_gd_violated ON gate_decisions USING GIN (violated_dimensions);  -- WORM
trigger CREATE TRIGGER trg_gate_decisions_worm     BEFORE UPDATE OR DELETE ON
gate_decisions     FOR EACH ROW EXECUTE FUNCTION enforce_append_only();
```

3.4 evidence_source_manifest Table

The `evidence_source_manifest` table records the complete set of RAG Vault chunks that contributed to a given Evidence Package. Each row captures the chunk's text hash at retrieval time (enabling retrospective verification that the chunk was not modified after retrieval), its gate stamp, classification, and rank position. This table supports the "source traceability" dimension of compliance review.

```
CREATE TABLE evidence_source_manifest (     manifest_entry_id       UUID
PRIMARY KEY DEFAULT gen_random_uuid(),     evidence_package_id     UUID NOT
NULL REFERENCES evidence_packages(evidence_package_id),     chunk_id
UUID NOT NULL,     source_id               UUID NOT NULL,     source_type
VARCHAR(64) NOT NULL,     source_uri              TEXT,     chunk_text_hash
CHAR(64) NOT NULL,  -- SHA-256 of chunk text at retrieval time
chunk_classification    SMALLINT NOT NULL,     gate_stamp              UUID
NOT NULL,       -- gate_decision_id that cleared this chunk
confidence_band         VARCHAR(8) NOT NULL,     vector_distance
NUMERIC(8,6),     rank_position           INTEGER NOT NULL,     committed_at
TIMESTAMPTZ NOT NULL DEFAULT NOW() );  CREATE INDEX idx_esm_package ON
evidence_source_manifest (evidence_package_id); CREATE INDEX idx_esm_source
ON evidence_source_manifest (source_id, committed_at DESC);  CREATE TRIGGER
trg_evidence_source_manifest_worm     BEFORE UPDATE OR DELETE ON
evidence_source_manifest     FOR EACH ROW EXECUTE FUNCTION
enforce_append_only();
```

3.5 quarantine_records Table

The `quarantine_records` table records every Gate 530 QUARANTINE decision. The `status` field is the one controlled exception to strict append-only semantics in the audit schema — it may be transitioned from `PENDING` to a resolved state via the `resolve_quarantine()` stored procedure, which is a `SECURITY DEFINER` procedure that temporarily suspends the WORM trigger for the duration of the status update only, and which itself writes an audit event to `quarantine_resolution_log`.

```
CREATE TABLE quarantine_records (     quarantine_id           UUID PRIMARY
KEY DEFAULT gen_random_uuid(),     trace_id                UUID NOT NULL,
tenant_id               UUID NOT NULL REFERENCES tenant_registry(tenant_id),
gate_decision_id        UUID NOT NULL REFERENCES
gate_decisions(gate_decision_id),     quarantine_reason       TEXT NOT NULL,
dimension_scores        JSONB NOT NULL,     status
VARCHAR(16) NOT NULL DEFAULT 'PENDING'                                  CHECK
(status IN ('PENDING', 'CLEARED', 'BLOCKED', 'EXPIRED')),     created_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(),     resolved_at             TIMESTAMPTZ,
resolver_id             UUID,     resolver_notes          TEXT,     expiry_at
TIMESTAMPTZ NOT NULL   -- Default: created_at + 24 hours );  CREATE INDEX
idx_qr_tenant_status ON quarantine_records (tenant_id, status, created_at
DESC);  CREATE TRIGGER trg_quarantine_records_worm     BEFORE UPDATE OR
DELETE ON quarantine_records     FOR EACH ROW EXECUTE FUNCTION
enforce_append_only();
```

Controlled UPDATE Exception — resolve_quarantine() Procedure

The `resolve_quarantine()` stored procedure is the only authorized path for updating a `quarantine_records` row. It is declared `SECURITY DEFINER` and uses a session-local GUC (`ios.worm_override`) to communicate the authorized override context to the trigger. This pattern ensures the override is always accompanied by an audit INSERT to `quarantine_resolution_log` within the same transaction.

```
-- Exception: status field may be updated via this dedicated stored procedure
only. -- The procedure itself writes an audit event — override is always
audited. CREATE OR REPLACE PROCEDURE resolve_quarantine(     p_quarantine_id
UUID,     p_resolution    VARCHAR(16),     p_resolver_id   UUID,     p_notes
TEXT ) LANGUAGE plpgsql SECURITY DEFINER AS $$ BEGIN     -- Disable WORM
trigger for this transaction only (controlled exception)     SET LOCAL
ios.worm_override = 'quarantine_resolution';     UPDATE quarantine_records
SET status       = p_resolution,         resolved_at  = NOW(),
resolver_id  = p_resolver_id,         resolver_notes = p_notes     WHERE
quarantine_id = p_quarantine_id       AND status = 'PENDING';     -- Write
resolution audit event (separate INSERT — always executed)     INSERT INTO
quarantine_resolution_log     VALUES (gen_random_uuid(), p_quarantine_id,
p_resolution,             p_resolver_id, p_notes, NOW()); END; $$;
```

3.6 merkle_roots Table

The `merkle_roots` table stores the signed Merkle root computed over each time-windowed batch of Evidence Package records. Roots are published to DNS TXT records for independent third-party verification. The `published_to_dns` flag is set by the Evidence Fabric after successful DNS publication. Rows are never updated or deleted; the `published_to_dns` field is the only field that appears to warrant an update path — it is handled by setting this field at initial insert once DNS publication is confirmed synchronously within the Evidence Fabric signing transaction.

```
CREATE TABLE merkle_roots (     root_id                 UUID PRIMARY KEY
DEFAULT gen_random_uuid(),     window_start            TIMESTAMPTZ NOT NULL,
window_end              TIMESTAMPTZ NOT NULL,     record_count
INTEGER NOT NULL,     merkle_root_hash        CHAR(64) NOT NULL,   -- SHA-256
hex of Merkle root     ed25519_signature       CHAR(88) NOT NULL,
signing_key_id          UUID NOT NULL,     published_to_dns        BOOLEAN
NOT NULL DEFAULT FALSE,     committed_at            TIMESTAMPTZ NOT NULL
DEFAULT NOW() );  CREATE TRIGGER trg_merkle_roots_worm     BEFORE UPDATE OR
DELETE ON merkle_roots     FOR EACH ROW EXECUTE FUNCTION
enforce_append_only();
```

Section 4 — Operational Table Schemas

Operational tables support the live IOS+ inference workflow. Unlike audit tables, some operational tables permit UPDATE operations (e.g., object TTL expiry, tenant status transitions). WORM triggers are applied selectively where append-only semantics are required for compliance reasons.

4.1 ios_signing_keys Table (Key Registry — Publication Location 1)

The `ios_signing_keys` table is the authoritative on-database registry of all Ed25519 signing keys used by the Evidence Fabric. Keys are never deleted from this registry — the full rotation history must be retained for retrospective signature verification.

The `enforce_append_only()` trigger is applied to DELETE operations only (UPDATE is permitted to support `revoked_at` field population on key revocation).

```
CREATE TABLE ios_signing_keys (     key_id                  UUID PRIMARY KEY
DEFAULT gen_random_uuid(),     public_key_bytes        BYTEA NOT NULL,
-- 32-byte Ed25519 public key     public_key_b64url       CHAR(43) NOT NULL
UNIQUE,    -- Base64url encoding     public_key_fingerprint  CHAR(64) NOT NULL
UNIQUE,    -- SHA-256 hex of public_key_bytes     purpose
VARCHAR(32) NOT NULL CHECK (purpose IN ('evidence_signing', 'gate_signing')),
tenant_id               UUID,                          -- NULL = global key
created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),     activates_at
TIMESTAMPTZ NOT NULL,     expires_at              TIMESTAMPTZ NOT NULL,
revoked_at              TIMESTAMPTZ,     rotation_predecessor_key_id UUID
REFERENCES ios_signing_keys(key_id),     created_by_operator     VARCHAR(256)
NOT NULL );  -- Append-only: keys are never deleted from this registry CREATE
TRIGGER trg_ios_signing_keys_worm     BEFORE DELETE ON ios_signing_keys
FOR EACH ROW EXECUTE FUNCTION enforce_append_only();  -- Function to get
current active signing key CREATE OR REPLACE FUNCTION
get_active_signing_key(p_tenant_id UUID DEFAULT NULL) RETURNS
ios_signing_keys LANGUAGE sql STABLE AS $$     SELECT * FROM ios_signing_keys
WHERE (tenant_id = p_tenant_id OR tenant_id IS NULL)     AND revoked_at IS
NULL     AND activates_at <= NOW()     AND expires_at > NOW()     ORDER BY
activates_at DESC     LIMIT 1; $$;
```

4.2 objects Table (Primary Operational Store)

The `objects` table is the primary operational data store for IOS+ inference payloads. It stores the JSONB payload alongside its compliance classification, vector embedding, and gate reference. The compliance-first index design means the most common access patterns — compliance-dimension-filtered retrieval within a tenant — are served by covering indexes without heap fetches for the most common projected columns.

```
CREATE TABLE objects (     object_id               UUID PRIMARY KEY DEFAULT
gen_random_uuid(),     tenant_id               UUID NOT NULL REFERENCES
tenant_registry(tenant_id),      -- Content     payload                 JSONB
NOT NULL,     payload_hash            CHAR(64) NOT NULL,      -- SHA-256 of
JSON-serialized payload     source_id               UUID,     source_type
VARCHAR(64),      -- Compliance dimensions (primary index)
classification_level    SMALLINT NOT NULL CHECK (classification_level BETWEEN
0 AND 4),     compliance_flags        INTEGER[] NOT NULL DEFAULT '{}',
-- Vector     vector_embedding        vector(1536),            -- text-
embedding-3-small dimensions      -- Gate reference     gate_decision_id
UUID REFERENCES gate_decisions(gate_decision_id),      -- Evidence
evidence_hash           CHAR(64),               -- ed25519_signature of
associated Evidence Package      -- Retention     tenant_encryption_key_id
UUID NOT NULL,     ttl                     TIMESTAMPTZ,     retain_until
TIMESTAMPTZ NOT NULL,      -- Temporal     created_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(),     indexed_at              TIMESTAMPTZ
```

```
NOT NULL DEFAULT NOW() );  -- Primary compliance index CREATE INDEX
idx_objects_compliance ON objects     (classification_level, tenant_id)
INCLUDE (object_id, payload_hash, created_at);  CREATE INDEX
idx_objects_flags ON objects USING GIN (compliance_flags); CREATE INDEX
idx_objects_vector ON objects USING hnsw (vector_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64); CREATE INDEX idx_objects_tenant_time ON
objects (tenant_id, created_at DESC); CREATE INDEX idx_objects_source ON
objects (source_id, tenant_id);
```

4.3 tenant_registry Table

The `tenant_registry` table is the authoritative tenant directory. Every foreign key referencing `tenant_id` throughout the schema traces back to this table. The `active_regulations` and `jurisdiction_flags` arrays are used at tenant onboarding to initialize the tenant's `regulatory_profiles` baseline.

```
CREATE TABLE tenant_registry (     tenant_id              UUID PRIMARY KEY
DEFAULT gen_random_uuid(),     tenant_name             VARCHAR(256) NOT NULL,
tenant_slug             VARCHAR(64) NOT NULL UNIQUE,     active_regulations
TEXT[] NOT NULL DEFAULT '{}',     jurisdiction_flags      TEXT[] NOT NULL
DEFAULT '{}',     max_classification_level SMALLINT NOT NULL DEFAULT 2,
data_residency_regions  TEXT[] NOT NULL DEFAULT '{}',     quarantine_queue_id
UUID,     onboarded_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
status                  VARCHAR(16) NOT NULL DEFAULT 'ACTIVE'
CHECK (status IN ('ACTIVE', 'SUSPENDED', 'OFFBOARDING')) );
```

4.4 regulatory_profiles Table

The `regulatory_profiles` table stores versioned regulatory threshold configurations per tenant. Each version is immutable once activated — the `superseded_at` timestamp marks when a version was replaced by a newer version, but the row is never deleted. This ensures that any historical Evidence Package can be re-evaluated against the exact regulatory profile version that was active at the time of the original evaluation, which is a prerequisite for audit reproducibility.

```
CREATE TABLE regulatory_profiles (     profile_id             UUID PRIMARY
KEY DEFAULT gen_random_uuid(),     tenant_id               UUID NOT NULL
REFERENCES tenant_registry(tenant_id),     profile_version         INTEGER
NOT NULL,     active_regulations      TEXT[] NOT NULL,
dimension_thresholds    JSONB NOT NULL,  -- {dimension_id: {monitor: float,
warn: float, block: float}}     jurisdiction_flags      TEXT[] NOT NULL,
sector_overlays         TEXT[] NOT NULL DEFAULT '{}',     created_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(),     activated_at            TIMESTAMPTZ,
superseded_at           TIMESTAMPTZ,    -- Set when a newer version is
```

```
activated     UNIQUE (tenant_id, profile_version) );  -- Profiles are never
deleted: superseded profiles retained for audit reproducibility CREATE
TRIGGER trg_regulatory_profiles_worm     BEFORE DELETE ON regulatory_profiles
FOR EACH ROW EXECUTE FUNCTION enforce_append_only();
```

Section 5 — Vector Corpus Tables (RAG Vault)

The RAG Vault corpus tables store the document chunks and their vector embeddings that power the retrieval-augmented generation pipeline. These tables are written by the `rag_writer` role (ingestion pipeline) and read by the `rag_reader` role (retrieval service). HNSW indexes on `vector_embedding` columns provide approximate nearest-neighbor retrieval at production scale. See EB Doc 5 for retrieval query patterns and RRF fusion logic.

5.1 rag_sources Table

The `rag_sources` table is the root record for each ingested document or data source. Every `rag_chunks` row traces back to exactly one `rag_sources` row. The `chunk_count` field is maintained by the ingestion pipeline (not a trigger-maintained aggregate) to support ingestion progress monitoring.

```
CREATE TABLE rag_sources (     source_id               UUID PRIMARY KEY
DEFAULT gen_random_uuid(),     tenant_id               UUID NOT NULL
REFERENCES tenant_registry(tenant_id),     source_uri              TEXT NOT
NULL,     source_type             VARCHAR(64) NOT NULL,  -- 'document',
'database', 'api', 'stream'     classification_level    SMALLINT NOT NULL,
jurisdiction_flags      TEXT[] NOT NULL DEFAULT '{}',     ingested_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(),     last_refreshed_at       TIMESTAMPTZ
NOT NULL DEFAULT NOW(),     ttl                     TIMESTAMPTZ,
chunk_count             INTEGER NOT NULL DEFAULT 0,     status
VARCHAR(16) NOT NULL DEFAULT 'ACTIVE' );
```

5.2 rag_chunks Table

The `rag_chunks` table stores the individual text chunks extracted from each source, their SHA-256 hashes, compliance classifications, and 1536-dimension vector embeddings. The HNSW index is built with higher `ef_construction` than the `objects` table to prioritize recall for the RAG retrieval workload. The `UNIQUE (source_id, chunk_index)` constraint prevents duplicate chunk ingestion from idempotent ingestion pipeline retries.

```
CREATE TABLE rag_chunks (     chunk_id                UUID PRIMARY KEY
DEFAULT gen_random_uuid(),     source_id               UUID NOT NULL
REFERENCES rag_sources(source_id),     tenant_id               UUID NOT NULL
REFERENCES tenant_registry(tenant_id),      chunk_text              TEXT NOT
NULL,     chunk_hash              CHAR(64) NOT NULL,          -- SHA-256 of
chunk_text (UTF-8)     chunk_index             INTEGER NOT NULL,
-- Position within source     token_count             INTEGER NOT NULL,
-- Compliance     classification_level    SMALLINT NOT NULL,
compliance_flags        INTEGER[] NOT NULL DEFAULT '{}',      -- Vector
vector_embedding        vector(1536) NOT NULL,      -- Temporal
indexed_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),     ttl
TIMESTAMPTZ,      UNIQUE (source_id, chunk_index) );  CREATE INDEX
idx_rag_chunks_vector ON rag_chunks     USING hnsw (vector_embedding
vector_cosine_ops)     WITH (m = 16, ef_construction = 128);  CREATE INDEX
idx_rag_chunks_classification ON rag_chunks     (classification_level,
tenant_id, indexed_at DESC);  CREATE INDEX idx_rag_chunks_source ON
rag_chunks (source_id, chunk_index);
```

Section 6 — Partitioning Strategy

6.1 evidence_packages Partitioning

For deployments generating more than 10 million Evidence Package records per month, the `evidence_packages` table must be converted to a declaratively partitioned table. The recommended strategy is **range partitioning on `committed_at` (monthly partitions)** with **hash sub-partitioning on `tenant_id` (8 sub-partitions per month)**. This two-level strategy delivers two benefits

simultaneously: partition pruning eliminates non-relevant months from compliance queries that carry a date range filter; hash sub-partitioning distributes write load across 8 physical segments per month, preventing hot-spot write contention on the current-month partition.

```
-- Convert evidence_packages to declarative range-partitioned table CREATE
TABLE evidence_packages (     -- (all columns as defined in Section
3.1)     ... ) PARTITION BY RANGE (committed_at);  -- Monthly partition
example — May 2026 CREATE TABLE evidence_packages_2026_05     PARTITION OF
evidence_packages     FOR VALUES FROM ('2026-05-01') TO ('2026-06-01')
PARTITION BY HASH (tenant_id);  -- Hash sub-partitions (8 per month) CREATE
TABLE evidence_packages_2026_05_h0     PARTITION OF evidence_packages_2026_05
FOR VALUES WITH (MODULUS 8, REMAINDER 0);  CREATE TABLE
evidence_packages_2026_05_h1     PARTITION OF evidence_packages_2026_05
FOR VALUES WITH (MODULUS 8, REMAINDER 1);  -- ... repeat for REMAINDER 2
through 7  -- Partition management: new monthly partitions are created by the
-- cos_admin maintenance procedure run on the 25th of each month. --
Partition creation is a schema migration (EB Doc 7 Section 2).
```

ℹ Partition Pruning Behavior

PostgreSQL 16 partition pruning is enabled by default (`enable_partition_pruning = on`). Compliance queries that include a `committed_at` range predicate will automatically skip non-relevant month partitions. Queries that omit a date range predicate will scan all partitions — query patterns without a date range filter should be reviewed and rejected in application code review.

6.2 rag_chunks Partitioning

The `rag_chunks` table is partitioned by **HASH on `tenant_id` (16 partitions)**. Unlike the time-series evidence_packages partitioning, the RAG corpus grows primarily along the tenant dimension — each tenant's corpus is independent and grows at its own rate. Hash partitioning on `tenant_id` distributes the HNSW index into 16 independent sub-indexes, each covering approximately 1/16 of the total corpus. This has a critical performance consequence for HNSW retrieval: each sub-index is

smaller, resulting in faster `ef_search` traversal and lower memory pressure per index scan.

```
CREATE TABLE rag_chunks (     -- (all columns as defined in Section
5.2)     ... ) PARTITION BY HASH (tenant_id);  -- 16 hash partitions CREATE
TABLE rag_chunks_h0  PARTITION OF rag_chunks FOR VALUES WITH (MODULUS 16,
REMAINDER 0); CREATE TABLE rag_chunks_h1  PARTITION OF rag_chunks FOR VALUES
WITH (MODULUS 16, REMAINDER 1); CREATE TABLE rag_chunks_h2  PARTITION OF
rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER 2); CREATE TABLE
rag_chunks_h3  PARTITION OF rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER
3); CREATE TABLE rag_chunks_h4  PARTITION OF rag_chunks FOR VALUES WITH
(MODULUS 16, REMAINDER 4); CREATE TABLE rag_chunks_h5  PARTITION OF
rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER 5); CREATE TABLE
rag_chunks_h6  PARTITION OF rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER
6); CREATE TABLE rag_chunks_h7  PARTITION OF rag_chunks FOR VALUES WITH
(MODULUS 16, REMAINDER 7); CREATE TABLE rag_chunks_h8  PARTITION OF
rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER 8); CREATE TABLE
rag_chunks_h9  PARTITION OF rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER
9); CREATE TABLE rag_chunks_h10 PARTITION OF rag_chunks FOR VALUES WITH
(MODULUS 16, REMAINDER 10); CREATE TABLE rag_chunks_h11 PARTITION OF
rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER 11); CREATE TABLE
rag_chunks_h12 PARTITION OF rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER
12); CREATE TABLE rag_chunks_h13 PARTITION OF rag_chunks FOR VALUES WITH
(MODULUS 16, REMAINDER 13); CREATE TABLE rag_chunks_h14 PARTITION OF
rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER 14); CREATE TABLE
rag_chunks_h15 PARTITION OF rag_chunks FOR VALUES WITH (MODULUS 16, REMAINDER
15);  -- HNSW indexes must be created on each partition individually. --
Partition-level indexes are built in parallel during initial ingestion. --
RAG Vault queries each partition independently and merges results -- with RRF
(Reciprocal Rank Fusion) — see EB Doc 5 Section 3.
```

Section 7 — Migration Procedures

7.1 Migration Framework

All COS+ schema changes are managed via **Flyway versioned migrations**. The following conventions are mandatory for all schema changes:

- **Migration naming convention:** `V{version}__{description}.sql` — e.g., `V20260522_001__add_quarantine_resolution_log.sql`. The version prefix uses ISO date format (YYYYMMDD) followed by a three-digit sequence number.

- **Idempotency:** Migrations must use `IF NOT EXISTS` / `IF EXISTS` guards where supported. DDL that cannot be made idempotent must document the pre-condition that guarantees safe re-run.
- **Breaking migrations — two-phase requirement:** Any migration that removes a column, changes a column type, renames a table, or removes an index that application code may depend on must be executed as a two-phase migration: Phase 1 makes the backward-compatible addition (e.g., new column added alongside old); Phase 2 removes the deprecated element after all services have been deployed against Phase 1 for a minimum of one release cycle.
- **Document synchronization:** EB Doc 3 must be updated in the same pull request as the migration file. A PR that contains a migration but does not update this document will be rejected at review.

7.2 Migration Review Requirements

The following migration categories require elevated review and additional verification steps before merging:

Migration Category	Required Reviewers	Additional Requirements
Audit table DDL (any column addition, constraint change, or index modification)	Engineering Lead + Security Lead	Tested against production data snapshot in staging; WORM verification test (Section 7.3) must pass post-migration
Role definitions or privilege changes	Engineering Lead + Security Lead	Privilege matrix in EB Doc 3 Section 2.2 must be updated; explicit REVOKE list re-verified
WORM trigger modification or removal	Engineering Lead + Security Lead + VP Engineering	Requires documented security exception; WORM verification test must pass post-

Migration Category	Required Reviewers	Additional Requirements
		deployment
Signing key table modifications	Engineering Lead + Security Lead	Rollback procedure must be documented; key rotation integrity test required
Partitioning structure changes	Engineering Lead + DBA	Partition pruning behavior verified in staging; query plan regression test required
All other migrations	Engineering Lead (or delegate)	Standard PR review; rollback procedure documented in migration comments

7.3 WORM Verification Test

The following SQL script must be executed after every migration that touches any audit table, WORM trigger, or role definition. It is also included in the CI/CD pipeline as a post-deployment gate. A failure in this test is a **deploy blocker** — the deployment must be rolled back immediately.

```
-- Run after every migration to verify WORM enforcement is intact DO $$
DECLARE v_package_id UUID; BEGIN     -- Insert a test record     INSERT INTO
evidence_packages (         trace_id, tenant_id, query_hash,
model_fingerprint,         gate_decision_id, gate_decision_state,
classification_level,         compliance_flags, signing_key_id,
jcs_canonical_hash,         ed25519_signature, ingestion_timestamp,
l4_timestamp,         gate_timestamp, tenant_encryption_key_id, retain_until
)     VALUES (         gen_random_uuid(),         '00000000-0000-0000-0000-
000000000000'::UUID,         repeat('a', 64), 'test',
gen_random_uuid(), 'PASS', 0, '{}',         gen_random_uuid(), repeat('b',
64), repeat('c', 88),         NOW(), NOW(), NOW(),         gen_random_uuid(),
NOW() + INTERVAL '1 year'     )     RETURNING evidence_package_id INTO
v_package_id;      -- Attempt UPDATE — must raise exception     BEGIN
UPDATE evidence_packages         SET output_hash = repeat('d', 64)
WHERE evidence_package_id = v_package_id;         RAISE EXCEPTION 'WORM TEST
FAILED: UPDATE succeeded on evidence_packages — trigger not active';
EXCEPTION WHEN restrict_violation THEN         RAISE NOTICE 'WORM TEST
PASSED: UPDATE correctly rejected on evidence_packages';     END;      --
Attempt DELETE — must raise exception     BEGIN         DELETE FROM
evidence_packages         WHERE evidence_package_id = v_package_id;
RAISE EXCEPTION 'WORM TEST FAILED: DELETE succeeded on evidence_packages —
trigger not active';     EXCEPTION WHEN restrict_violation THEN         RAISE
NOTICE 'WORM TEST PASSED: DELETE correctly rejected on evidence_packages';
```

```
END;      RAISE NOTICE 'WORM VERIFICATION COMPLETE: All append-only
constraints confirmed active'; END; $$;
```

> ⚠ **Test Record Cleanup**
>
> The test INSERT above inserts a record into `evidence_packages` with a synthetic `tenant_id` of all zeros. Because WORM prevents deletion, this test record will persist. The staging and CI environments must use a dedicated `tenant_id` reserved for test records (`00000000-0000-0000-0000-000000000000`), and compliance queries must exclude this sentinel tenant ID. This record must never be created against a production database instance.

Section 8 — Performance Tuning Configuration

8.1 postgresql.conf Recommendations

The following `postgresql.conf` settings are tuned for a COS+ production workload on a 64 GB RAM, NVMe-backed node. Settings are annotated with the rationale specific to the COS+ workload. Deviations from these settings require documentation and approval from the Engineering Lead.

```
# ================================================== # COS+ Production
postgresql.conf — SMEPro Technologies # Target node: 64GB RAM, NVMe storage,
PostgreSQL 16.x # ==================================================  #
Memory shared_buffers = 16GB                  # 25% of 64GB RAM — PostgreSQL
convention effective_cache_size = 48GB            # OS page cache estimate —
used by planner work_mem = 64MB                       # Per sort/hash op;
200 connections * 64MB = 12.8GB max maintenance_work_mem = 2GB             #
HNSW index builds require significant working memory  # Connections
max_connections = 200                 # PgBouncer handles application
connections upstream                                    # 200 = DBA +
```

```
monitoring + PgBouncer pool ceiling  # WAL and replication wal_level =
replica                    # Required for WAL streaming replication
max_wal_senders = 10                  # 2 standbys + 8 headroom for
pgbackup/monitoring wal_keep_size = 2GB                    # Retain WAL for
standby re-sync after brief disconnect hot_standby = on
# Allow read queries on standby nodes synchronous_commit = remote_write
# Audit tables: WAL must be acknowledged by standby
# before evidence_packages INSERT returns to caller  # Checkpointing
checkpoint_completion_target = 0.9    # Spread checkpoint writes over 90% of
checkpoint interval  # Storage (NVMe-optimized) effective_io_concurrency =
200        # NVMe supports high I/O concurrency; enables bitmap heap scan
random_page_cost = 1.1               # NVMe near-sequential — planner should
prefer index scans  # Archiving (15-minute RPO) archive_mode = on
archive_command = 'aws s3 cp %p s3://cos-wal-archive/wal/%f' archive_timeout
= 900                  # Force WAL segment switch every 15 minutes (RPO = 15
min)
```

8.2 pgvector HNSW Tuning

HNSW index behavior is controlled by the `hnsw.ef_search` GUC, which can be set at session level before issuing retrieval queries. The following values are recommended for each retrieval context:

Context	ef_search Value	Rationale
Standard RAG retrieval (real-time inference)	40	Balances recall (~95%) against latency; acceptable for production inference path
High-precision audit retrieval (compliance review)	100	Maximizes recall at cost of additional latency; acceptable for non-real-time compliance queries
Index build — rag_chunks (high recall corpus)	N/A (build param: m=16, ef_construction=128)	Higher ef_construction at build time improves long-term recall; build is offline batch
Index build — objects (lower corpus size)	N/A (build param: m=8, ef_construction=64)	Smaller objects corpus does not require the same graph density as rag_chunks

```
-- Set before real-time RAG retrieval queries SET hnsw.ef_search = 40;  --
Set before compliance audit retrieval queries SET hnsw.ef_search = 100;
```

ℹ HNSW Memory at Query Time

HNSW graph traversal uses working memory proportional to `ef_search`. With `work_mem = 64MB`, setting `ef_search` above 200 on the 1536-dimension corpus may cause working memory spills. Do not raise `ef_search` beyond 100 without first increasing `work_mem` in the session or verifying memory headroom on the node.

Section 9 — Backup & Replication

9.1 WAL Streaming Replication (15-Minute RPO)

COS+ operates a **primary/hot-standby configuration** using PostgreSQL WAL streaming replication. The standby node is configured with `hot_standby = on`, enabling read-only queries for monitoring and compliance analytics without impacting the primary. The critical synchronization guarantee for audit integrity is:

- **`synchronous_commit = remote_write`** is set globally. This ensures that any `INSERT` into an audit table (evidence_packages, gate_decisions, etc.) does not return success to the calling service until the WAL record has been written to the standby's receive buffer. This prevents a scenario where a primary node failure between the INSERT acknowledgment and WAL propagation results in a lost audit record with no client-visible error.
- **WAL archiving to S3-compatible object storage** is configured with `archive_timeout = 900` (15-minute segment cadence). This provides the 15-minute RPO guarantee independent of standby availability — even if the

standby is unavailable, WAL segments archived within the last 15 minutes allow point-in-time recovery to within 15 minutes of failure.

- **Recovery procedures** for WAL-based restore, standby promotion, and point-in-time recovery are documented in EB Doc 6 Section 4. Engineers must not attempt manual recovery without consulting EB Doc 6 first.

```
-- Primary: standby connection entry in pg_hba.conf # TYPE  DATABASE   USER
ADDRESS         METHOD host    replication replicator   10.0.1.0/24
scram-sha-256  -- Primary: replication slot (prevents WAL deletion before
standby consumes) SELECT
pg_create_physical_replication_slot('cos_standby_slot');  -- Standby:
recovery.conf / postgresql.conf (PostgreSQL 16 integrated) primary_conninfo =
'host=10.0.1.10 port=5432 user=replicator password=<<vault>>
application_name=cos_standby' primary_slot_name = 'cos_standby_slot'
restore_command = 'aws s3 cp s3://cos-wal-archive/wal/%f %p'
```

9.2 Logical Replication for Analytics

For deployments where compliance analytics workloads (aggregate queries across large date ranges in `evidence_packages` and `gate_decisions`) would conflict with the production primary's query performance, a **logical replication publication** allows a dedicated read-only analytics replica to be maintained without impacting the primary's HNSW retrieval and audit write workloads.

```
-- Create the analytics publication on the primary CREATE PUBLICATION
cos_analytics     FOR TABLE evidence_packages, gate_decisions;  -- On the
analytics replica: create the subscription CREATE SUBSCRIPTION
cos_analytics_sub     CONNECTION 'host=10.0.1.10 dbname=cosplus
user=replicator password=<<vault>>'     PUBLICATION cos_analytics;  -- Note:
the analytics replica receives only the published tables. -- It does not
receive rag_chunks, objects, or other operational tables. -- Analytics
queries against rag_chunks must run on the primary or -- a dedicated hot
standby, not the analytics replica.
```

> **ℹ Logical Replication and WORM**
>
> The analytics replica receives only `INSERT` operations from the logical replication stream (since UPDATE and DELETE are blocked by WORM triggers on the primary and therefore never appear in the WAL). The WORM triggers must also be installed on the analytics replica to prevent direct modification of replicated rows via local connections. Replica WORM trigger installation is included in the

analytics replica deployment playbook (EB Doc 6 Section 5).

Section 10 — Document Lineage

EB Doc 3 is not a standalone document. The schema definitions it contains are the implementation of cryptographic contracts, architectural decisions, and pipeline specifications defined in adjacent EB documents. The following lineage table maps each cross-document dependency.

Source Document	Dependency	Implemented In (EB Doc 3)
EB Doc 2 — Cryptographic Specification	Defines the Ed25519 signing protocol, JCS canonicalization hash, and Base64url signature encoding	Fields `ed25519_signature CHAR(88)`, `jcs_canonical_hash CHAR(64)`, `signing_key_id` in `evidence_packages`, `gate_decisions`, and `ios_signing_keys`
EB Doc 1 — Internal Architecture	Defines the IOS+ Middleware Engine, Evidence Fabric, and RAG Vault as distinct services with distinct database access roles	Role definitions in Section 2 (`ios_app`, `audit_writer`, `rag_reader`, `rag_writer`); connection pool configuration in Section 8.1
EB Doc 5 — RAG Vault Pipeline	Defines retrieval query patterns against the RAG corpus, including RRF fusion across partitions and compliance-filtered ANN search	Tables `rag_chunks` and `rag_sources` in Section 5; HNSW index configuration in Section 8.2; partition structure in Section 6.2
EB Doc 6 — Operations Runbook	Defines WAL-based recovery procedures, standby promotion, partition maintenance scheduling, and WORM incident response	WAL configuration in Section 9.1; replication slot setup in Section 9.1; WORM verification test in Section 7.3

Appendix A — Complete Table Index

This appendix provides a consolidated reference of all tables, indexes, and triggers defined in this document.

Table Name	Category	Primary Key	WORM	Indexes	Trigger(s)
`evidence_packages`	Audit	`evidence_package_id`	UPDATE + DELETE	idx_ep_compliance (GIN), idx_ep_classification, idx_ep_tenant_time, idx_ep_gate_decision, idx_ep_trace	trg_evidence_packages_worm
`gate_decisions`	Audit	`gate_decision_id`	UPDATE + DELETE	idx_gd_tenant_time, idx_gd_state, idx_gd_violated (GIN)	trg_gate_decisions_worm
`evidence_source_manifest`	Audit	`manifest_entry_id`	UPDATE + DELETE	idx_esm_package, idx_esm_source	trg_evidence_source_manifest_worm
`quarantine_records`	Audit	`quarantine_id`	UPDATE + DELETE (controlled exception via procedure)	idx_qr_tenant_status	trg_quarantine_records_worm
`merkle_roots`	Audit	`root_id`	UPDATE + DELETE	PK only	trg_merkle_roots_worm
`ios_signing_keys`	Operational	`key_id`	DELETE	UNIQUE on	trg_ios_sign

Table Name	Category	Primary Key	WORM	Indexes	Trigger(s)
			only	public_key_b64url, public_key_fingerprint	ing_keys_worm
`objects`	Operational	`object_id`	None	idx_objects_compliance (covering), idx_objects_flags (GIN), idx_objects_vector (HNSW), idx_objects_tenant_time, idx_objects_source	None
`tenant_registry`	Operational	`tenant_id`	None	UNIQUE on tenant_slug	None
`regulatory_profiles`	Operational	`profile_id`	DELETE only	UNIQUE (tenant_id, profile_version)	trg_regulatory_profiles_worm
`rag_sources`	Vector Corpus	`source_id`	None	PK only	None
`rag_chunks`	Vector Corpus	`chunk_id`	None	idx_rag_chunks_vector (HNSW), idx_rag_chunks_classification, idx_rag_chunks_source; UNIQUE (source_id, chunk_index)	None

Appendix B — Role Privilege Quick Reference Card

One-page summary of all role privileges for use by database engineers, security auditors, and NDA-cleared evaluators during schema review. For full privilege grant and REVOKE DDL, refer to Section 2.

Role	Login	Operational Tables	Audit Tables	Vector Corpus Tables	CREATE ROLE	Primary Consumer
`ios_app`	Yes	SELECT, INSERT	**NONE**	**NONE**	No	IOS+ Middleware Engine
`audit_writer`	Yes	**NONE**	INSERT only	**NONE**	No	Evidence Fabric signing service
`audit_reader`	Yes	SELECT	SELECT only	**NONE**	No	Compliance review API; external auditors
`rag_reader`	Yes	SELECT	**NONE**	SELECT only	No	RAG Vault retrieval service
`rag_writer`	Yes	**NONE**	**NONE**	INSERT, UPDATE	No	RAG Vault ingestion pipeline
`cos_admin`	Yes	ALL	ALL (WORM applies)	ALL	Yes	DBA / infrastructure operations only; pgaudit-logged

> ⚠ **WORM Applies to ALL Roles Including cos_admin**
>
> The privilege matrix above does not override WORM trigger enforcement. Even roles with ALL privileges on audit tables cannot execute UPDATE or DELETE on those tables. The trigger layer is independent of the privilege layer. There is no role, privilege, or connection attribute that bypasses the trigger for UPDATE/DELETE on audit tables. The only authorized exception path is the `resolve_quarantine()` SECURITY DEFINER procedure for `quarantine_records` status transitions.

Appendix C — Migration Log

This appendix records the version history of all schema changes applied to the COS+ Database. Engineers must add an entry to this log in the same pull request as the migration file and the corresponding EB Doc 3 update.

Migration ID	Date	Author	Description	Review By	WORM Test
`V20260522_001`	2026-05-22	SMEPro Engineering	Initial schema baseline: all tables, indexes, triggers, roles, and extensions as defined in EB Doc 3 v1.0	Engineering Lead + Security Lead	PASSED
(future entries)					

Appendix D — Glossary

Term	Definition
WORM	Write Once, Read Many. A data immutability model in which records can be inserted but never modified or deleted. In COS+, WORM is enforced at both the role privilege layer (no UPDATE/DELETE grants on audit roles) and the trigger layer (BEFORE UPDATE OR DELETE trigger raises restrict_violation). The trigger layer is the structural guarantee; the role layer is the operational enforcement.
WAL	Write-Ahead Log. PostgreSQL's transaction log, written before changes are applied to data files. WAL streaming replication transmits WAL records from a primary node to standby nodes in near-real-time, enabling hot standby failover and point-in-time recovery within the archive window.
HNSW	Hierarchical Navigable Small World. An approximate nearest-neighbor graph index algorithm implemented by pgvector. HNSW indexes support fast cosine similarity search over high-dimensional vectors (1536 dimensions in COS+). Index quality is controlled by `m` (graph connectivity) and `ef_construction` (build-time beam search width) parameters; retrieval quality is controlled by `ef_search` (query-time beam search width).
pgvector	A PostgreSQL extension that adds the `vector` data type and vector similarity search operators (cosine, L2, inner product) along with IVFFlat and HNSW index types. Required extension for all vector embedding columns and ANN retrieval in COS+.
Flyway	An open-source database schema migration framework that manages versioned SQL migration scripts. Flyway

Term	Definition
	tracks which migrations have been applied in a `flyway_schema_history` table and applies pending migrations in version order. COS+ uses Flyway's versioned migration model with ISO-date-prefixed migration file names.
pgaudit	A PostgreSQL extension that provides detailed session- and object-level audit logging beyond the standard PostgreSQL log. In COS+, pgaudit is configured to log all relation-level operations performed under the `cos_admin` role, producing an out-of-band audit trail that cannot be suppressed by the `cos_admin` role itself.
SECURITY DEFINER	A PostgreSQL function/procedure attribute that causes the routine to execute with the privileges of the user who defined it (the owner), rather than the user who calls it. In COS+, `resolve_quarantine()` is declared SECURITY DEFINER to allow the procedure to perform a controlled UPDATE on `quarantine_records` (setting the local override GUC) regardless of the caller's role privileges — while still producing a mandatory audit INSERT within the same transaction.
GIN Index	Generalized Inverted Index. A PostgreSQL index type optimized for multi-valued column types such as arrays and JSONB. In COS+, GIN indexes are used on `compliance_flags INTEGER[]` columns to enable fast containment queries (`compliance_flags @> ARRAY[dim_id]`) across the Evidence Package and object corpus.
Declarative Partitioning	PostgreSQL's built-in table partitioning model (introduced in PostgreSQL 10, matured in PostgreSQL 12+) in which partitions are defined as child tables of a parent table with `PARTITION BY RANGE/HASH/LIST` syntax. The query planner automatically prunes non-relevant partitions for queries with partition key predicates. COS+ uses declarative range partitioning (evidence_packages by committed_at)

Term	Definition
	and hash partitioning (rag_chunks by tenant_id).
RRF	Reciprocal Rank Fusion. A result merging algorithm that combines ranked result lists from multiple independent retrievals (e.g., one per rag_chunks partition) into a single unified ranking without requiring score normalization. RRF is used by the RAG Vault retrieval service to merge HNSW results across rag_chunks hash partitions. Defined in detail in EB Doc 5 Section 3.
JCS	JSON Canonicalization Scheme (RFC 8785). A deterministic JSON serialization algorithm that produces a stable byte sequence from a JSON object regardless of key insertion order or whitespace. COS+ uses JCS canonicalization to produce the `jcs_canonical_hash` field — the SHA-256 of the canonical bytes — before Ed25519 signing, ensuring the signature covers a deterministic representation of the Evidence Package payload. Defined in detail in EB Doc 2.
RPO	Recovery Point Objective. The maximum tolerable data loss window in a disaster recovery scenario, measured as the time between the last recoverable state and the point of failure. COS+ targets a 15-minute RPO, enforced by WAL segment archiving with `archive_timeout = 900` and synchronous WAL replication to a hot standby.